02 Inji | Build an Interoperability Playground for Verifiable Credentials

Inji Interoperability Playground: A Unified Sandbox for Verifiable Credential Interoperability Testing

Build a single, visual "playground" web app where anyone can issue, hold, present, and verify Verifiable Credentials across Inji Certify, Inji Wallet (mobile + web), and Inji Verify and cross-test them against other OpenID4VCI / OpenID4VP-compliant wallets, issuers, and verifiers to prove real interoperability, not just internal compatibility.

Complexity Level: High

High (multi-module orchestration), credential formats, cryptographic algorithm, standards compliance across OpenID4VCI / OpenID4VP / W3C VC, and cross-team wallet interoperability testing)

Overview

Inji Stack is a modular, open-source suite for the full verifiable-credential lifecycle: Inji Certify issues credentials, Inji Wallet(mobile app built on React Native with backend(Mimoto) in Java, and web app built on React with backend(Mimoto) in Java) stores and presents them, and Inji Verify validates and verifies them at the relying-party end. Each module already speaks open standards OpenID4VCI (draft 13) for issuance, OpenID4VP (draft 21–23) for presentation, and W3C VC 1.1/2.0, IETF SD-JWT VC and mDoc/mDL (ISO/IEC 18013-5) as credential formats but real-world digital identity ecosystems (eSignet, EUDI wallets, other national ID stacks) need proof that these modules interoperate cleanly with implementations outside of MOSIP itself, and that the three Inji modules interoperate smoothly with each other under varied configurations.

Inspired by community "VC interoperability playgrounds" (which let developers pick an issuer, a wallet, and a verifier from a matrix of implementations and run a live issue → hold → present → verify flow while inspecting every payload exchanged), this problem statement asks you to build the same experience purpose-built for the Inji ecosystem. The playground should let a user configure any combination of: an issuer (Inji Certify, or a mocked/external OpenID4VCI issuer), a wallet (Inji Wallet mobile via QR/deep-link, Inji Web, or a mocked external wallet), and a verifier (Inji Verify, or a mocked external OpenID4VP relying party) then run the flow end-to-end, visualising every protocol exchange, credential format, signature scheme, and failure mode along the way.

The Goal Is Two-Fold:

a. Give the Inji community and new integrators a hands-on way to understand and trust the stack's standards compliance.
b. Surface real interoperability gaps (format mismatches, proof-type incompatibilities, DID method gaps, QR size limits, etc.) that the core teams can act on.

Recommended Tech Stack

- **Frontend / Playground UI:** React + TypeScript (Next.js or Vite), Tailwind CSS reuse @mosip/react-inji-verify-sdk components (QRCodeVerification, OpenID4VPVerification) directly where possible.
- **Sequence/protocol visualisation:** Mermaid.js or a custom WebSocket-driven timeline to show live OpenID4VCI / OpenID4VP request-response exchanges (auth request, token, credential offer, vp_token, etc.).
- **Backend orchestrator (Playground BFF):** Node.js/Express or Spring Boot proxies calls to Mimoto, Inji Certify, and Inji Verify APIs and normalises responses for the UI.
- **Issuer module:** Inji Certify (Spring Boot, plugin-based) run via Docker Compose, using mock or MOSIP ID / eSignet-backed plugins.
- **Wallet modules:** Inji Wallet mobile (React Native, Mimoto BFF, Tuvali for offline BLE transfer, PixelPass for QR encode/decode) and Inji Web (React) for browser-based credential holding.
- **Verifier module:** Inji Verify (Spring Boot verify-service + React verify-ui), or the standalone Inji Verify SDK / react-inji-verify-sdk npm package for a lightweight custom relying-party.
- **Interop libraries:** inji-openid4vp (Kotlin/Swift) for wallet-side protocol handling, inji-vci-client for credential requests, mosip/pixelpass for QR compression.
- **Credential formats to cover:** W3C VC 1.1 / 2.0 (JSON-LD, Ed25519Signature2020 / RSA proofs), SD-JWT VC, and mso_mdoc (ISO 18013-5, mock mode).
- **Infra:** Docker Compose (mirroring Inji-Certify and Inji-Web local-setup patterns), Postgres/Redis as required by each module, ngrok/reverse-proxy for mobile-to-local testing.
- **Optional:** an external reference implementation (e.g. a Sphereon/Walt.id/DIF-style OpenID4VCI or OpenID4VP mock) to prove true cross-ecosystem interoperability, not just Inji-to-Inji.

Expected Tasks

Mandatory Tasks

1. Stand up the core Inji Stack locally/in the MOSIP Collab sandbox: Inji Certify (issuer), Mimoto + Inji Wallet or Inji Web (holder), and Inji Verify (verifier), connected end-to-end.
2. Build a playground UI with a 3-column "Issuer → Wallet → Verifier" selector, letting the user pick which implementation plays each role (starting with all three being Inji modules).
3. Implement a live "Issue a Credential" flow: trigger Inji Certify to issue a VC via OpenID4VCI to a selected wallet (Inji Wallet mobile via QR/deep-link, or Inji Web), and show the credential offer, token exchange, and issued credential in the UI.
4. Implement a live "Present & Verify" flow: from the wallet, share the credential via OpenID4VP (QR-based cross-device flow, or same-device deep-link) to Inji Verify, and display the verification result (Valid / Invalid / Valid-but-Expired).
5. Add a protocol inspector panel that shows the raw exchanged payloads at each step (authorization request, credential offer, vp_token, presentation_definition, verification result JSON) for developer transparency.
6. Support at least two credential formats end-to-end (e.g. W3C VC JSON-LD and SD-JWT VC) and clearly label which format is in use per test run.
7. Produce a pass/fail interoperability report per test run (issuer, wallet, verifier, format, proof type, outcome, error detail if failed).

Good-to-have Tasks

- Extend the wallet/verifier matrix to include at least one non-Inji, standards-compliant mock (e.g. a generic OpenID4VCI issuer or OpenID4VP verifier) to demonstrate true cross-ecosystem interoperability.
- Add mDoc/mDL (ISO/IEC 18013-5) as a third credential format, using Inji Certify's mock mDoc support.
- Visualise the OpenID4VP sequence as an animated/interactive Mermaid sequence diagram (Verifier ↔ Wallet ↔ Library) rather than a static log.
- Add a "replay & tamper" mode that lets users deliberately corrupt a VC/VP (wrong type, expired, bad signature) to demonstrate Inji Verify's negative-path handling.
- Build a historical dashboard of past interoperability runs with filters by module version, format, and outcome useful for regression tracking across Inji releases.
- Support the OpenID4VP Same-Device flow via deep link/Digital Credentials API in addition to the cross-device QR flow.

Bonus Tasks

- Automate the playground as a CI-style interoperability test suite (using inji-verify's api-test patterns) that can run nightly against the Collab environment and post a compatibility matrix.
- Contribute discovered interoperability gaps back as issues to the relevant mosip/inji-* GitHub repositories.
- Add telemetry/observability (reusing Inji's Obsrv/Druid pattern) to track success rates and latency per flow.
- Package the whole playground as a one-command Docker Compose bundle so any developer can spin it up locally in minutes.

Deliverables

- A working, deployed (or locally runnable via Docker Compose) Inji Interoperability Playground web application.
- Source code in a public GitHub repository with a clear README covering setup, architecture, and how to run each flow.
- A short demo video (3–5 minutes) walking through at least one full issue → hold → present → verify cycle with the protocol inspector visible.
- An interoperability report/matrix (markdown or in-app) summarising which issuer/wallet/verifier/format combinations were tested and their pass/fail outcome.
- A brief write-up of any real interoperability gaps or bugs discovered during testing, ideally filed as GitHub issues against the relevant Inji repositories.

Resources

Inji Documentation

- Inji Docs Overview & Features
- Inji Certify Overview
- Inji Verify Overview
- Inji Wallet (Mobile) Technical Overview
- Inji Web Local Setup
- OpenID4VP Integration Guide (Inji Verify SDK)
- Inji Resources Webinars & Deep Dives

GitHub Repositories

- Inji-wallet Mobile wallet (React Native)
- Inji-web Inji Web (browser wallet)
- Inji-certify Credential issuance (OpenID4VCI, plugin-based)
- Inji-verify Verifier UI, verify-service, SDK & api-test
- Inji-openid4vp OpenID4VP protocol library (Kotlin/Swift)
- mosip/react-inji-verify-sdk (npm) Ready-to-use React verification components
- mosip/inji-config Shared issuer & wallet configuration (mimoto-issuers-config.json)

MOSIP Collab Sandbox Environment(Reference Only)

The Collab sandbox lets you exercise the full Inji Credentialing Stack: Certify, Wallet, Web, and Verify without standing up every backend locally. It is the fastest path to a working end-to-end demo during the hackathon. This sandbox is a reference to test and try the existing Inji solution purely for understanding purposes.

- Collab Environment Guide (Inji Docs)
- Mimoto (BFF)
- eSignet
- Inji Certify (mock issuer, Collab)
- External VC Playground (for reference only) *(Note: Please use it only to understand the overall concept and user experience. Kindly avoid replicating it as-is.)*

Community & Support

- MOSIP Community Forum
- Inji Docs Home